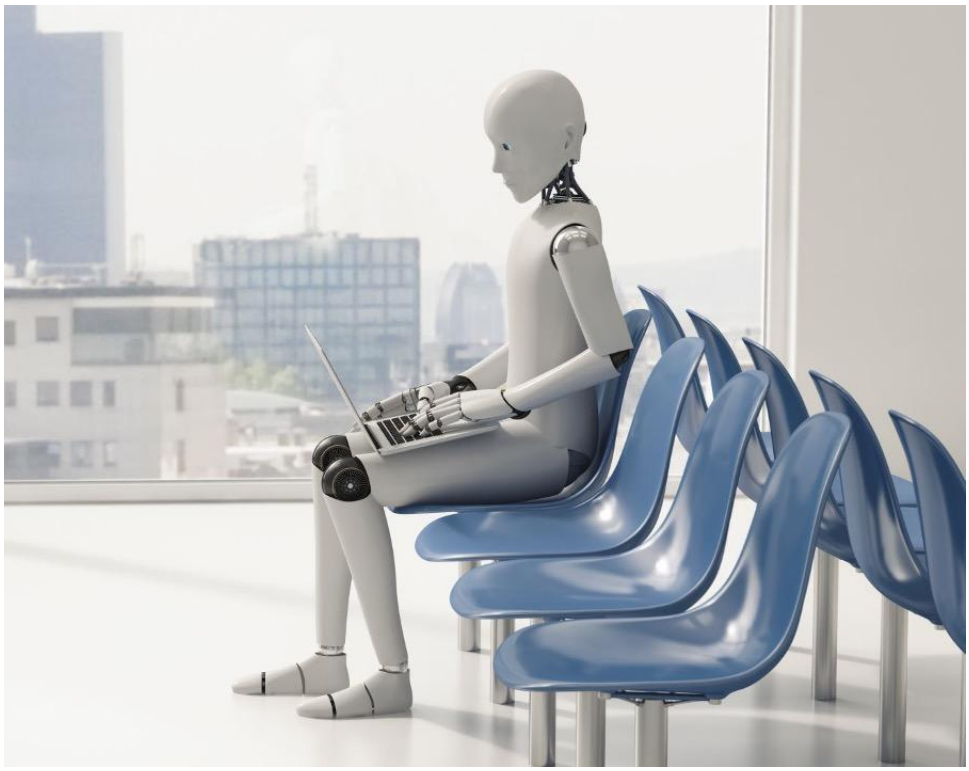

Assessment: Customer 360 Platform on Microsoft Fabric

Warehouse T-SQL Security (Stored Procedures, Cross-Warehouse, RLS, CLS) and Deployment & DevOps

Build a Customer 360 Platform with Fabric Warehouse, T-SQL Security and Deployment Pipelines

1. Business Scenario

A retail bank wants to build a single, governed view of every customer — a Customer 360 platform. Today the customer data lives in three disconnected systems: a CRM platform (customer master and interaction history), the core-banking system (financial transactions), and a compliance system (KYC records). Because these systems do not talk to each other, business, compliance, and relationship-management teams cannot see a unified, trustworthy picture of a customer.

Participants will land raw data from all three source systems into a single Bronze layer in a Fabric Lakehouse, curate it, and then serve it to business users through a Fabric Warehouse using T-SQL. Because the data carries sensitive PII (KYC document numbers, contact details) and must be segmented by relationship manager and region, the serving layer must enforce Row-Level Security (RLS) and Column-Level Security (CLS). Finally, the solution must be promoted across Development, Test, and Production workspaces using Git integration and Deployment Pipelines.

The platform should help the bank answer questions such as:

1. What does a single, unified profile of each customer look like across CRM, transactions, and KYC?
2. Which customers carry a high compliance risk while also showing high transaction value?
3. Can a relationship manager see only the customers belonging to their own region?
4. Are PII fields protected from analysts who do not have a need to see them?
5. How is the solution promoted safely from Dev to Test to Production?

2. Source System Overview

Participants will work with the following source extracts:

Source System	File	Description
CRM	`crm_customers.csv`	Customer master demographics, segment and assigned RM
CRM	`crm_interactions.csv`	Customer touchpoints across service channels
Transactions	`transactions.csv`	Core-banking financial transactions
KYC / Compliance	`kyc_records.csv`	KYC documents, verification status and risk rating
Reference	`relationship_managers.csv`	RM-to-region mapping used to drive RLS

3. Sample Dataset Structure

A. crm_customers.csv

Column	Description
`customer_id`	Unique customer ID (business key)
`full_name`	Customer full name
`email`	Email address (PII)

Column	Description
phone	Mobile number (PII)
date_of_birth	Date of birth (PII)
gender	Male / Female / Other
city	Customer city
region	North, South, East, West, Central
customer_segment	Retail, Premium, SME, Corporate
rm_id	Assigned relationship manager ID
marketing_consent	Yes / No
crm_created_date	Date customer record was created in CRM

B. crm_interactions.csv

Column	Description
interaction_id	Unique interaction ID
customer_id	Linked customer ID
interaction_date	Date of interaction
channel	Call Center, Email, Branch, Mobile App, Chatbot
interaction_type	Enquiry, Complaint, Service Request, Feedback
sentiment	Positive, Neutral, Negative
resolved_flag	Y / N

C. transactions.csv

Column	Description
transaction_id	Unique transaction ID
customer_id	Linked customer ID
account_id	Linked account ID
transaction_date	Date of transaction
transaction_type	Debit, Credit, UPI, NEFT, IMPS, ATM
transaction_amount	Amount of transaction
currency	Transaction currency (e.g., INR)
transaction_status	Success, Failed, Pending
merchant_category	Retail, Travel, Utilities, Investment, Other

D. kyc_records.csv

Column	Description
kyc_id	Unique KYC record ID
customer_id	Linked customer ID
document_type	PAN, Aadhaar, Passport, Driving License
document_number	Document identifier (sensitive PII)

Column	Description
kyc_status	Verified, Pending, Rejected
risk_rating	Low, Medium, High
verified_date	Date KYC was verified
expiry_date	KYC / document expiry date

E. relationship_managers.csv

Column	Description
rm_id	Unique relationship manager ID
rm_name	Relationship manager name
rm_login	Workspace login / UPN used for RLS mapping
region	Region the RM is responsible for
branch_city	Primary branch city

4. Assignment Tasks

Task 1: Build the Customer 360 Bronze Layer (Lakehouse)

Objective

Land raw CRM, transactions, and KYC extracts into a single Bronze layer in a Fabric Lakehouse, with no business transformations applied.

Participant Activities

1. Create a Lakehouse named cust360_lakehouse and upload all five CSV files into the Files section.
2. Load each file into a Bronze Delta table in a common bronze schema/zone: `bronze_crm_customers`, `bronze_crm_interactions`, `bronze_transactions`, `bronze_kyc_records`, `bronze_relationship_managers`.
3. Add ingestion metadata columns to every Bronze table:
   - `source_system` (CRM / Transactions / KYC / Reference)
   - `source_file_name`
   - `ingestion_timestamp`
   - `pipeline_run_id`
4. Preserve the raw structure and original data types of each source; do not deduplicate or clean at this stage.

Expected Output

Bronze Table	Expected Result
bronze_crm_customers	Raw CRM customer master loaded with metadata
bronze_crm_interactions	Raw CRM interaction history loaded
bronze_transactions	Raw core-banking transactions loaded
bronze_kyc_records	Raw KYC / compliance records loaded

Bronze Table	Expected Result
`bronze_relationship_managers`	Raw RM reference data loaded

Task 2: Choose the Serving Layer — Lakehouse vs Warehouse

Objective

Decide where the curated, business-facing Customer 360 model should be served from, and justify the choice of a Fabric Warehouse versus a Lakehouse for downstream serving.

Participant Activities

1. Compare the Lakehouse SQL endpoint and the Fabric Warehouse against the requirements of this scenario (multi-table T-SQL joins, stored procedures, transactional writes, RLS / CLS, and BI serving).
2. Complete the decision matrix below and record a one-paragraph recommendation in your documentation.
3. Create a Fabric Warehouse named `cust360_warehouse` to act as the governed serving layer for the curated Customer 360 model.

Lakehouse vs Warehouse Decision Matrix

Capability / Need	Lakehouse (SQL endpoint)	Warehouse
Primary write engine	Spark / Dataflow / notebooks	T-SQL (INSERT, UPDATE, DELETE)
Read-only or read-write SQL	Read-only over Delta	Full read-write T-SQL
Stored procedures	Not supported	Supported
Best fit for	Data engineering, ML, bronze/silver	Governed serving, BI, security
RLS / CLS enforcement	Limited	Native T-SQL RLS & CLS

Expected Output

A documented recommendation that the curated, security-governed serving model is built in cust360_warehouse, while raw and engineering layers remain in cust360_lakehouse.

Task 3: Build the Serving Warehouse with T-SQL Stored Procedures

Objective

Use T-SQL stored procedures in the Warehouse to build and refresh the curated Customer 360 serving tables.

Participant Activities

1. Create a curated schema (for example `c360`) in the Warehouse.
2. Build the unified profile table `c360.dim_customer_360` with the columns described below, sourced from the curated CRM, transactions, and KYC data.
3. Author a parameterized stored procedure `c360.usp_build_customer_360` that (re)loads the table. Use a load pattern of your choice (TRUNCATE + INSERT, or MERGE) and accept an `@as_of_date` parameter.

4. Author a second stored procedure `c360.usp_refresh_risk_summary` that aggregates transaction value and KYC risk per customer.
5. Execute the procedures and confirm the serving tables are populated.

Target Table: c360.dim_customer_360

Column	Description
`customer_id`	Customer business key
`full_name`	Customer name
`region`	Customer region
`customer_segment`	Customer segment
`rm_id`	Assigned relationship manager
`total_transactions`	Count of successful transactions
`total_transaction_amount`	Total value of successful transactions
`kyc_status`	Latest KYC status
`risk_rating`	Latest KYC risk rating
`email`	Email (PII — protected later via CLS)
`document_number`	KYC document number (PII — protected via CLS)

Task 4: Cross-Warehouse / Cross-Item Querying

Objective

Query data that lives across the Lakehouse SQL endpoint and the Warehouse in a single T-SQL statement using three-part naming, without copying data between them.

Participant Activities

1. Ensure cust360_lakehouse and cust360_warehouse are in the same workspace so both appear in the SQL query scope.
2. Write a single cross-item query that joins a curated Warehouse table with a Bronze/curated table read from the Lakehouse SQL endpoint, using three-part naming: `[item].[schema].[table]`.
3. Produce a result that enriches the Warehouse Customer 360 rows with the most recent interaction sentiment held in the Lakehouse.
4. Save the query in your sql_scripts folder and capture the result in a screenshot.

Expected Output

A working cross-warehouse query (for example, `SELECT ... FROM cust360_warehouse.c360.dim_customer_360 w JOIN cust360_lakehouse.dbo.bronze_crm_interactions i ON ...`) that returns enriched Customer 360 rows.

Task 5: Implement Row-Level Security (RLS) and Column-Level Security (CLS)

Objective

Protect the Customer 360 serving layer so that relationship managers see only their region's customers (RLS), and analysts cannot read PII columns (CLS).

Participant Activities — Row-Level Security

1. Create a mapping table `c360.rm_region_map` linking each RM login (UPN) to a region.
2. Create an inline table-valued function as a security predicate that returns rows only where the customer's region matches the region mapped to the current user (use `USER_NAME()` / `SUSER_SNAME()`).
3. Create a `SECURITY POLICY` with a `FILTER PREDICATE` on `c360.dim_customer_360` and enable it.

Participant Activities — Column-Level Security

1. Create a database role `analyst_role` representing general analysts.
2. Grant column-level SELECT on the non-sensitive columns of `c360.dim_customer_360`, and DENY SELECT on the PII columns `email`, `phone`, `date_of_birth`, and `document_number`.

Example Security Rules

Rule ID	Rule Description
R1	An RM may read only customers whose region matches their mapped region.
R2	Compliance role may read all regions (no filter predicate applied).
R3	Analyst role is denied SELECT on email, phone, date_of_birth and document_number.
R4	PII columns must never appear in BI datasets built for general analysts.
R5	Security policy must remain enabled after deployment to every stage.

Task 6: Deployment & DevOps (Overview)

Objective

Place the solution under source control and promote it through Development, Test, and Production using Fabric Deployment Pipelines, with awareness of Service Principals, Azure DevOps CI/CD and the Fabric REST APIs.

Participant Activities — Hands-on

1. Connect the Development workspace to a Git repository (Azure DevOps Repos or GitHub) using Fabric Git integration.
2. Commit the Lakehouse, Warehouse and supporting items, then make a small change and sync it to demonstrate commit and update direction.
3. Create a Deployment Pipeline with three stages — Development, Test, Production — and assign a workspace to each.
4. Deploy from Development to Test, and configure at least one deployment rule (for example, repointing a connection or parameter between stages).

5. Business Questions to Answer

Using the curated Warehouse serving layer, participants should answer:

1. Which region contributes the highest total successful transaction value?

2. Which customers are both high transaction value and High KYC risk rating?
3. When querying as a North-region RM, are only North-region customers returned (proving RLS)?
4. Does an analyst-role SELECT fail on PII columns while succeeding on allowed columns (proving CLS)?
5. Which customer segment has the largest share of unresolved complaints (cross-item query)?
6. How many customers have an expired or rejected KYC status?
7. After deploying Dev → Test, are the security policy and CLS grants still enforced in Test?

6. Deliverables

Deliverable	Description
Bronze Tables	Raw CRM, transactions, KYC tables with metadata columns
Serving Decision Note	Lakehouse vs Warehouse decision matrix and recommendation
Warehouse & Stored Procedures	Curated c360 schema with build / refresh procedures
Cross-Warehouse Query	Single T-SQL query across Lakehouse and Warehouse
RLS Implementation	Mapping table, predicate function, security policy
CLS Implementation	Role, column grants
DevOps Evidence	Git commit history
Architecture Diagram	Source → Bronze → Warehouse serving → BI flow

7. Suggested Architecture Diagram

```
CRM  +  Transactions  +  KYC  (CSV extracts)
        |
        v
Bronze Layer  (cust360_lakehouse)
Single bronze zone, raw tables + ingestion metadata
        |
        v
Curated / Serving Layer  (cust360_warehouse)
T-SQL stored procedures build dim_customer_360 & risk summary
        |
        v
Governance Layer
Row-Level Security (region)  +  Column-Level Security (PII)
        |
        v
Power BI / SQL Reporting   (governed Customer 360)
        |
        v
DevOps:  Git integration
```

8. Evaluation Rubric

Criteria	Weightage
Bronze ingestion of CRM, transactions, KYC with metadata	15%
Lakehouse vs Warehouse decision and serving model	10%

Criteria	Weightage
T-SQL stored procedures building the Customer 360 model	20%
Cross-warehouse / cross-item query	10%
Row-Level Security and Column-Level Security implementation	25%
Git integration	15%
Final insights and documentation	5%